

With this module, I learned how to implement JWT-based authentication with full-stack integration, combining backend security with client-side validation and automated browser testing. Adding the login and registration flow deepened my understanding of how frontend and backend work together to create a secure user experience. The client-side validation (checking password length, email format, and password matching) provides immediate user feedback, while the backend validation with Pydantic ensures no invalid data enters the system regardless of how the request is made. Storing JWT tokens in localStorage and including them in Authorization headers showed me how stateless authentication works across requests.

Playwright E2E tests were particularly valuable for verifying the complete user flow from browser interaction to server response. Testing both valid scenarios (successful registration and login) and invalid scenarios (short passwords, mismatched passwords, wrong credentials) ensured the UI properly displays error messages and success states. These browser-based tests complement the API integration tests by verifying the actual user experience, not just endpoint functionality.

The assignment presented several significant challenges. Initially, the registration form displayed "[object Object]" instead of error messages because the JavaScript tried to read HTML error responses as JSON. After fixing the error handling, I encountered a 500 Internal Server Error caused by the deprecated aioredis library. Switching the import from "import aioredis" to "import redis.asyncio as aioredis" resolved the Python 3.12 compatibility issue. However, the ModuleNotFoundError for redis persisted because the Docker container was not rebuilt after adding redis to requirements.txt. Running docker-compose build with the no-cache flag forced a complete rebuild and installed the package. Finally, I added a success message display on registration with a 10-second delay before redirecting, allowing enough time to capture screenshots for assignment submission. This experience reinforced the importance of proper error handling in JavaScript, managing deprecated dependencies, and understanding Docker's caching when updating requirements.